



INGENIERÍA EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

Título

Entendiendo la estructura de FASTAPI y Pydantic

Profesor:

Miguel Angel Cardona Contreras

Materia:

Desarrollo para Dispositivos Inteligentes

Alumno:

Alvares Salazar Erick Sidrac

Tijuana, BC

Mi proyecto de FASTAPI

Introducción:

En este proyecto se exploró el uso de apis para demostrar cómo funciona el hasheo de contraseñas, esto con la finalidad de aprender sobre la verificación de la integridad e identidad de los datos sin necesidad de conocer el dato original. También se exploró el concepto de “SAL” que en grandes rasgos se muestra como caracteres al azar dentro de un dato hasheado para aumentar la complejidad del dato como un “ruido” dentro de la operación que la hace indescifrable (hasta el momento).

Desarrollo:

Ya que anteriormente vimos el desarrollo de apps simples con fastapi mostraremos directamente el tema que compete a esta asignatura como el hasheo de la información mediante una función que utiliza la librería passlib y simplemente toma un dato y le pasa como función el hasheo:



```
security.py X
core > security.py > ...
1  # core/security.py
2
3  from passlib.context import CryptContext
4
5
6  # Configuración para Argon2
7  pwd_context = CryptContext(
8      schemes=["argon2"],
9      deprecated="auto",
10     argon2__time_cost=2,
11     argon2__memory_cost=10240,
12     argon2__parallelism=2,
13 )
14
15
16 def get_password_hash(password: str) -> str:
17     """Hashea la contraseña usando Argon2."""
18     return pwd_context.hash(password)
19
20
```

Llamamos a la función de hasheo y agregamos la contraseña hasheada como los datos del usuario para que al ingresarlos en la api esta operación se realice antes de entrar a la BD

```

    )
    # Crear el usuario (en producción, hashear la contraseña con bcrypt)
    db_user = User(
        username=user_data.username,
        email=user_data.email,
        hashed_password=get_password_hash(user_data.password), # Aquí deber
        is_active=True
    )

```

Ingresamos un usuario por medio del endpoint correspondiente para verificar que el hasheo funcione.

POST /api/v1/users/ Create User

Crea un nuevo usuario en la base de datos.

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```

{
  "username": "string1",
  "email": "user@example1.com",
  "password": "Aserejel"
}

```

Curl

```

curl -X 'POST' \
  'http://127.0.0.1:8000/api/v1/users/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "string1",
    "email": "user@example1.com",
    "password": "Aserejel"
  }'

```

Request URL

http://127.0.0.1:8000/api/v1/users/

Server response

Code Details

201

Response body

```

{
  "id": 2,
  "username": "string1",
  "email": "user@example1.com",
  "is_active": true,
  "created_at": "2026-09-16T02:59:25.706662"
}

```

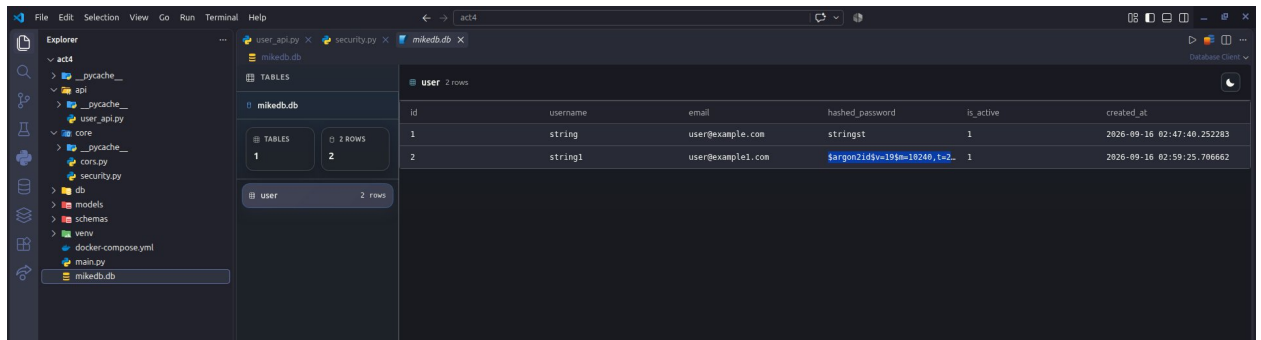
Response headers

```

access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
content-length: 116
content-type: application/json
date: Wed, 16 Sep 2026 02:59:25 GMT
server: uvicorn
vary: Origin

```

Por último comprobamos en la base de datos que el hasheo se haya realizado de forma correcta:



Conclusión:

Durante la práctica aprendimos la importancia del encriptado de información, no solo para proteger a los usuarios de ataques de software, sino que también debemos de proteger ciertos datos de cualquier persona que no sean ellos mismos. Incluso nosotros como desarrolladores no debemos tener acceso a toda la información ya que puede llevar tanto a problemas legales como de confianza con los usuarios.